

A Comparative Study of Logistic Regression, Multinomial Naive Bayes, and Decision Trees for Sentiment Analysis in Dating App Reviews

Arthur Decker

*Dept. of Electrical Engineering
University of North Carolina at Charlotte
Charlotte, NC, USA*

Adam Funak

*Dept. of Electrical Engineering
University of North Carolina at Charlotte
Charlotte, NC, USA*

Abstract—This project compares three machine learning models—Logistic Regression, Multinomial Naive Bayes, and a Decision Tree—on a dataset of over 680,000 user reviews of Tinder, Bumble, and Hinge. The goal is to classify each review as positive or negative based on its text. The pipeline cleans the text, converts it into a bag-of-words feature matrix, reduces the feature count using chi-square selection, and then trains and tests each model with an 80/20 split. We replaced the PCA step from our project proposal with chi-square selection because PCA does not work well on sparse text features and is not compatible with Naive Bayes. Logistic Regression performed best with an F1-score of 0.9008, followed by Naive Bayes at 0.8891 and the Decision Tree at 0.8056. The Logistic Regression coefficients also produce a list of the words most associated with positive and negative sentiment, which makes the model easy to interpret.

Index Terms—sentiment analysis, text classification, logistic regression, naive bayes, decision tree, chi-square, dating apps

I. INTRODUCTION

Dating apps like Tinder, Bumble, and Hinge generate a very large amount of written user feedback. The dataset used in this project contains more than 680,000 reviews. Reading that many reviews by hand is not realistic nor desirable, so app developers need an automated way to tell the difference between a positive review and a negative one. This is a binary text classification problem.

The challenge with dating app reviews is that the language is specific to the person. Words like “catfish,” “ghosted,” “bot,” and “cooked” show up often in this dataset and carry strong sentiment, but a general sentiment model would probably miss them. Training a model directly on this data lets it learn the vocabulary that actually matters for these apps.

This paper has three goals. First, we build a complete classification pipeline and compare three different models on the dataset. Second, we replace the PCA reduction step from our original proposal with chi-square feature selection, which is the standard approach for sparse text data. Third, we report not just accuracy but also F1-score, confusion matrices, a per-app breakdown, and a list of the most important words used by the best model.

The rest of the paper is organized as follows. Section II reviews related work. Section III describes the dataset, pre-

processing, models, and evaluation. Section IV presents the results. Section V concludes.

II. RELATED WORK

Guzman and Maalej [1] studied how to extract feature opinions (such as battery life or UI bugs) from app store reviews. Their work showed that short app reviews carry enough signal for sentiment analysis even though they are short and noisy. This is relevant to our project because dating app reviews are also short and informal.

Whitlow et al. [2] performed sentiment analysis on mental health app reviews. They found that domain-specific vocabulary performed better than general sentiment word lists. This supports our decision to let the model learn its own vocabulary from the dating app dataset rather than using a pretrained sentiment list.

The three models we compare are well known in text classification. McCallum and Nigam [3] described the multinomial Naive Bayes model used here. Joachims [4] showed that linear models work well on sparse high-dimensional text features, which is why we expect Logistic Regression to perform well. Decision Trees [5] are less common for text but are useful as an interpretable non-linear baseline.

Our project proposal originally used PCA for dimensionality reduction. After receiving reviewer feedback, we changed this to chi-square feature selection. PCA is not the standard choice for bag-of-words data because it requires dense input, which loses the efficiency of sparse text matrices. Chi-square selection [6], [7] keeps the matrix sparse and also keeps the original word identities, which is needed for the feature importance analysis later in the paper.

III. METHODOLOGY

A. Dataset

The dataset is the Dating App Reviews Dataset from Kaggle [9]. It contains the review text, a 1–5 star rating, and the app name. The raw dataset has 681,994 rows.

We applied the following filters in order:

- 1) Dropped rows with a missing review.
- 2) Kept only ratings between 1 and 5.

- 3) Removed all 3-star reviews because they are neutral and add noise to a binary classification task.
- 4) Labeled 1–2 star reviews as Negative (0) and 4–5 star reviews as Positive (1).

After these steps and after removing reviews that were empty after cleaning, the dataset contained 617,752 reviews. The class balance was close to even, with 304,068 negative reviews and 319,690 positive reviews. The dataset is dominated by Tinder reviews (482,520 raw rows) compared to Bumble (93,537) and Hinge (47,701).

B. Text Preprocessing

Each review was cleaned with a simple function that lowercased the text, replaced anything that was not a letter or whitespace with a space, and collapsed extra whitespace. Empty reviews after cleaning were dropped. We did not apply stemming or lemmatization because informal misspellings like “awsm” and “kool” carry useful sentiment information that stemming would remove.

C. Vectorization

We used scikit-learn’s CountVectorizer [8] to convert the text into a bag-of-words feature matrix. The settings were:

- English stop words removed.
- `min_df=5`, so a word had to appear in at least five reviews to be kept.
- `max_features=20,000`.
- Unigrams only.

The final vectorized matrix had a shape of (617,752, 17,073) with 4,325,111 non-zero entries. The vocabulary ended up smaller than the 20,000 cap because `min_df` had already filtered out the rare words.

D. Feature Selection (Chi-Square)

We reduced the feature count from 17,073 to 3,000 using scikit-learn’s `SelectKBest` with the chi-square test. The chi-square test scores each word based on how strongly it is associated with the class label. Words that appear roughly equally in positive and negative reviews get low scores and are dropped. Words that strongly favor one class are kept.

a) *Why chi-square instead of PCA:* Our project proposal used PCA. The reviewer pointed out that PCA is not the right choice for bag-of-words text. There are three reasons:

- 1) **Sparsity.** The bag-of-words matrix is very sparse. PCA requires centering, which forces the matrix to become dense. This wastes memory and slows things down.
- 2) **Interpretability.** PCA combines words into linear components, so the new features are no longer real words. Chi-square keeps the original words, which is needed for the feature importance chart.
- 3) **Naive Bayes compatibility.** Multinomial Naive Bayes requires non-negative integer features. PCA produces real-valued components with negative numbers, which Naive Bayes cannot use directly. Chi-square keeps the original counts.

After feature selection, the matrix shape was (617,752, 3,000).

TABLE I
TEST SET PERFORMANCE OF THE THREE MODELS.

Model	Acc.	Prec.	Rec.	F1	Train (s)
Logistic Regression	0.8974	0.8880	0.9139	0.9008	11.5
Multinomial Naive Bayes	0.8867	0.8867	0.8916	0.8891	0.1
Decision Tree	0.7773	0.7257	0.9052	0.8056	34.6

E. Train/Test Split

We split the data 80/20 using scikit-learn’s `train_test_split` with `random_state=42` and stratified sampling on the label. The training set had 494,201 reviews and the test set had 123,551 reviews.

F. Models

We trained three models, each chosen to represent a different style of classification.

Logistic Regression. A linear model with L2 regularization and the liblinear solver. The L2 penalty prevents the model from relying too heavily on any one word. We used $C = 1.0$ and `max_iter=1000`.

Multinomial Naive Bayes. A probabilistic model that assumes each word in a review contributes independently to the sentiment. This assumption is not really true in practice, but the model is known to work well on text [3]. We used the default Laplace smoothing.

Decision Tree. A non-linear model that splits the data based on one feature at a time. We used `max_depth=25` and `min_samples_leaf=50` to keep the tree from overfitting.

G. Evaluation Metrics

We report four metrics on the test set:

- **Accuracy:** fraction of reviews classified correctly.
- **Precision:** of the reviews predicted positive, the fraction that were actually positive.
- **Recall:** of the reviews that were actually positive, the fraction the model found.
- **F1-score:** the harmonic mean of precision and recall. This is our main metric.

We also report the confusion matrix for each model and a per-app breakdown for the best model.

IV. RESULTS AND ANALYSIS

A. Overall Performance

Table I shows the results for all three models on the held-out test set of 123,551 reviews.

Logistic Regression had the best F1-score at 0.9008. Multinomial Naive Bayes was a close second at 0.8891 and trained much faster (0.1 seconds versus 11.5 seconds). The Decision Tree was the worst at 0.8056 and also took the longest to train at 34.6 seconds.

The reason Logistic Regression won is that the bag-of-words feature space is very high-dimensional and roughly linearly separable, which is the setting where linear models do well.

TABLE II
TOP 10 NEGATIVE TRIGGER WORDS.

Word	Coef.	Word	Coef.
rubbish	-3.639	worst	-3.192
worst	-3.634	pathetic	-3.038
ruined	-3.413	shite	-2.990
useless	-3.347	poor	-2.920
garbage	-3.336	trash	-3.297

TABLE III
TOP 10 POSITIVE TRIGGER WORDS.

Word	Coef.	Word	Coef.
dope	+3.870	lit	+3.165
superb	+3.719	amazing	+3.007
excellent	+3.689	awsm	+2.968
awesome	+3.517	downside	+2.950
awesome	+3.466	kool	+2.937

The L2 penalty also prevents the model from overfitting to specific words.

Naive Bayes did almost as well even though its independence assumption is not really true. This is a known result for text classification: the model often works fine in practice even when its assumptions are violated [3].

The Decision Tree had high recall (0.9052) but low precision (0.7257). This means it predicted “positive” too often. The reason is that a single tree can only check one word at a time at each split, so it cannot combine signals from many words the way a linear model can.

B. Confusion Matrices

Figure 1 shows the confusion matrices for all three models. Logistic Regression and Naive Bayes have similar error patterns, with around 7,200 false positives and 5,400–6,800 false negatives. The Decision Tree is much worse on false positives with 21,543 negative reviews labeled as positive. This matches the low precision number in Table I.

C. Feature Importance

Because chi-square selection keeps the original words, the coefficients learned by Logistic Regression can be matched directly to specific words. A large positive coefficient means the word strongly predicts a positive review and a large negative coefficient means the word strongly predicts a negative review. Figure 2 shows the top 15 words for each class, and Tables II and III list the top 10 along with their coefficients.

There are three things worth pointing out about these results.

First, the negative words are mostly general insults (rubbish, worst, useless, garbage, trash) rather than the dating-specific words we predicted in the proposal (scam, bot, paywall). Those terms still exist in the model and contribute to its decisions, but they did not make the top 15 by coefficient size. The general insults dominate because they appear consistently across all three apps.

TABLE IV
PER-APP PERFORMANCE OF LOGISTIC REGRESSION ON THE TEST SET.

App	Test rows	Accuracy	F1
Tinder	95,567	0.8993	0.9031
Bumble	18,520	0.8869	0.8804
Hinge	9,464	0.8982	0.9120

Second, the model learned that misspelled versions of strong sentiment words are also strong signals. Words like “awesome,” “awsm,” “kool,” and “wrost” all show up in the top trigger words. This is why we did not apply stemming. A stemmer would have collapsed these into one token and lost useful information.

Third, “downside” shows up as a positive trigger, which seems wrong at first. Looking at the data, this word usually appears in phrases like “no downside” or “only downside is...,” which are actually positive contexts. The unigram bag-of-words model cannot see this kind of phrase structure. This is a limitation we discuss in the conclusion.

D. Per-App Performance

Table IV shows the per-app performance of Logistic Regression on the test set.

The model performs similarly across all three apps. Hinge had the highest F1 at 0.9120 even though it had the smallest test set and contributed the least training data. Bumble was the lowest at 0.8804. The fact that performance is consistent across apps suggests the model has learned a general sentiment vocabulary rather than one that only works on Tinder.

E. Discussion

A few points are worth highlighting.

The change from PCA to chi-square was important. PCA is not designed for sparse text data and would have made the pipeline slower while producing features that could not be interpreted as words. Chi-square fits the data better and made the feature importance analysis possible.

The final F1 of 0.9008 is higher than the 75–80% accuracy range we projected in the proposal. This is mainly because the dataset is very large (over 617,000 reviews after filtering), which gives the linear models enough data to learn solid weights for thousands of words.

The main remaining error is that the models cannot understand phrase structure. The “no downside” issue is a clear example. Adding bigram features would help with this, and using a transformer-based encoder would help even more.

F. Limitations

There are three main limitations. First, we only used unigram features, so the model cannot represent negation. “Not good” is the same bag of words as “good not.” Second, sarcasm is hard for any bag-of-words model. Third, the dataset only includes users who chose to write a review, which biases the data toward strong opinions.

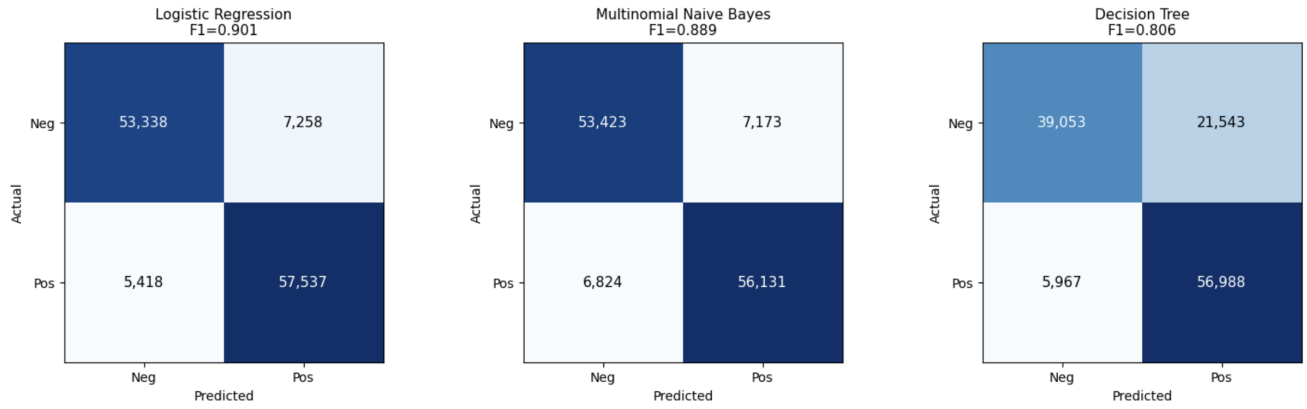


Fig. 1. Confusion matrices for Logistic Regression, Multinomial Naive Bayes, and Decision Tree on the test set of 123,551 reviews.

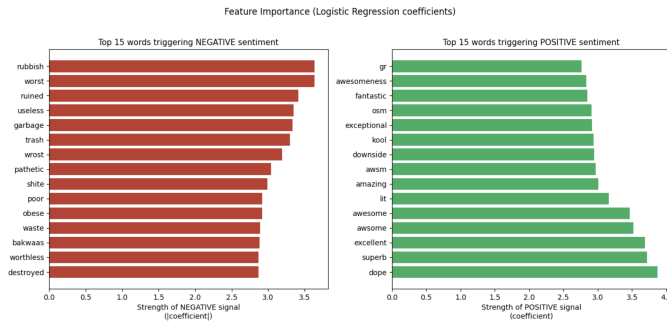


Fig. 2. Top 15 words triggering negative and positive sentiment, ranked by Logistic Regression coefficient.

V. CONCLUSION AND FUTURE WORK

We compared three classical machine learning models—Logistic Regression, Multinomial Naive Bayes, and a Decision Tree—on a dataset of 617,752 dating app reviews. After replacing the PCA step from the project proposal with chi-square feature selection, the final pipeline produced an F1-score of 0.9008 with Logistic Regression, 0.8891 with Naive Bayes, and 0.8056 with the Decision Tree. The Logistic Regression coefficients give an interpretable list of the words that most strongly predict each class, including both expected words like “worst” and “excellent” and informal ones like “awsm” and “kool.” The model performed consistently across Tinder, Bumble, and Hinge.

Three things would improve the project in future work. First, adding bigram features would let the model handle phrases like “no downside,” which the current unigram model gets wrong. Second, replacing the bag-of-words representation with a transformer-based encoder like DistilBERT would handle sarcasm and longer phrases much better, although it would be slower and harder to interpret. Third, extending the binary classification into a multiclass setup that routes reviews to specific teams (trust and safety, billing, bug reports) would make the output more directly useful to app developers.

CODE AVAILABILITY

The full code, data preparation script, and result files are included in the project submission. GitHub repository: https://github.com/KingKaiserr/ML_Final_Project

REFERENCES

- [1] E. Guzman and W. Maalej, “How do users like this feature? A fine grained sentiment analysis of app reviews,” in *Proc. 22nd IEEE Int. Requirements Engineering Conf. (RE)*, 2014, pp. 153–162.
- [2] R. J. Whitlow et al., “User reviews of depression app features: Sentiment analysis,” *JMIR Formative Research*, vol. 5, no. 11, 2021.
- [3] A. McCallum and K. Nigam, “A comparison of event models for naive Bayes text classification,” in *Proc. AAAI Workshop on Learning for Text Categorization*, 1998, pp. 41–48.
- [4] T. Joachims, “Text categorization with support vector machines: Learning with many relevant features,” in *Proc. 10th European Conf. on Machine Learning (ECML)*, 1998, pp. 137–142.
- [5] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*, Belmont, CA: Wadsworth, 1984.
- [6] G. Forman, “An extensive empirical study of feature selection metrics for text classification,” *Journal of Machine Learning Research*, vol. 3, pp. 1289–1305, 2003.
- [7] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*, Cambridge University Press, 2008.
- [8] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [9] S. Kriplani, “Dating App Reviews Dataset,” *Kaggle*, 2022. [Online]. Available: <https://www.kaggle.com/datasets/sidharthkriplani/datingappreviews>
- [10] The Pandas Development Team, “Pandas software library,” 2022. [Online]. Available: <https://pandas.pydata.org/>